Fee Flow and Splitter Review

April 16, 2026

Prepared for ZKsync

Conducted by:

Richie Humphrey (devtooligan)

Mayowa Olatunji (web3mio)

About the ZKsync Fee Flow and Splitter Review

ZKsync Era is a Layer 2 ZK rollup that uses cryptographic validity proofs to provide scalable and low-cost transactions on Ethereum. The Fee Flow and Splitter system enables on-chain interoperability fees and off-chain enterprise licensing fees to be gathered and converted to ZK programmatically. All ZK flows into a governance-controlled system that converts fees into ZK and directs it toward governance-defined targets such as staking rewards, token burn, and ecosystem funding.

The system consists of two core contracts deployed on ZKsync ERA. The Fee Flow contract implements a fixed-price auction mechanism where bidders exchange ZK for accumulated fee tokens, with the ZK forwarded to the Splitter contract. The Splitter contract then divides the received ZK between burning and distribution, where the distribution portion is further split and sent to each distribution contract based on its governance-defined allocation percentage.

About Offbeat Security

Offbeat Security is a boutique security company providing unique security solutions for complex and novel crypto projects. Our mission is to elevate the blockchain security landscape through invention and collaboration.

Summary & Scope

The src folder of the `zksync-fee-flow` repo was reviewed at commit 2c18543.

The following **2 contracts** were in scope:

- src/FeeFlow.sol
- src/Splitter.sol

Summary of Findings

The audit identified two low-severity issues, one gas optimization, and three informational findings across the Fee Flow and Splitter contracts. No critical or high-severity vulnerabilities were found. The low-severity findings relate to access control gaps in token recovery and missing initialization validation. The gas optimization addresses redundant storage reads, and the informational findings cover code quality improvements and deploy script hardening.

Identifier	Title	Severity	Fixed
L-01	`recover()` can extract actively claimable fee tokens	Low	From project team: "We aren't too worried about this as the dao will control the recover which will be heavily scrutinized"
L-02	Missing validation on token parameters in `initialize()`	Low	PR#44
I-01	Redundant balance check in claim loop	Informational	From project team: "We don't think this is an issue, because removing this check removes a revert when both the balance and minRequested are 0."
I-02	Unsafe downcast of distributor weights in deploy script	Informational	PR#45

I-03	Splitter lacks a recover function for stray tokens	Informational	PR#46
I-04	Distributor weights are unconstrained ratios instead of enforced percentages	Informational	Ack
G-01	Cache storage reads in `claim` to avoid redundant SLOADs	Gas	PR#42

Additional Recommendations

The spec requires explicit methods to `add`, `remove`, and `update` individual distributors, as well as `batch` variants of each. The implementation instead provides a single `setDistributors()` function that atomically replaces the entire distributor array. We consider this a safer design — it eliminates partial-update edge cases — but we do recommend updating the spec to reflect what was built.

Detailed Findings

Low Findings

[L-01] `recover()` can extract actively claimable fee tokens

Summary

The `recover()` function allows the default admin to sweep any token — including tokens that are actively on the claimable whitelist and expected to be available via auction. A low-cost guard would prevent accidental extraction of claimable tokens while preserving the intended use case of rescuing non-whitelisted assets.

Description

`FeeFlow` provides a `recover()` function gated on `DEFAULT_ADMIN_ROLE` that transfers any token to any address:

```
function recover(IERC20 _token, address _to, uint256 _amount) external {
    _revertIfNotDefaultAdmin();
    _token.safeTransfer(_to, _amount);
```

```
    emit Recovered(_token, _to, _amount);
}
```

There is no check preventing the admin from recovering tokens that are on the claimable whitelist. This means the admin could — intentionally or accidentally — drain fee tokens that claimers expect to be available via the fixed-price auction. The intended purpose of `recover()` is to rescue tokens that are not on the whitelist and therefore unreachable through `claim()`.

Recommendation

Consider adding a check that prevents recovering tokens which are currently on the claimable whitelist:

```
  function recover(IERC20 _token, address _to, uint256 _amount) external {
    _revertIfNotDefaultAdmin();
+   FeeFlowStorage storage $ = _getFeeFlowStorage();
+   if ($._claimableTokens[_token]) revert FeeFlow_TokenIsClaimable();
    _token.safeTransfer(_to, _amount);
    emit Recovered(_token, _to, _amount);
  }
```

An admin who genuinely needs to recover a whitelisted token can still do so by first calling `setClaimableToken(_token, false)` and then `recover()`. This two-step process documents intent and prevents accidental extraction.

[L-02] Missing validation on token parameters in `initialize()`

Summary

Both `FeeFlow.initialize()` and `Splitter.initialize()` validate admin and destination addresses but skip validation for their core token parameters. These tokens are write-once with no setter, so an invalid value at initialization permanently bricks the contract.

Description

Both `FeeFlow.initialize()` and `Splitter.initialize()` validate admin and destination addresses but skip this validation for their core token parameters.

In `FeeFlow.initialize()`, `_admin`, `_emergencyAdmin`, and `_destination` are all checked:

```
if (_admin == address(0)) revert FeeFlow_InvalidAddress();
if (_emergencyAdmin == address(0)) revert FeeFlow_InvalidAddress();
if (_destination == address(0)) revert FeeFlow_InvalidAddress();
```

But `_bidToken` (line 129) is stored without validation:

```
$._bidToken = _bidToken;
```

Similarly, `Splitter.initialize()` validates `_admin` and `_emergencyAdmin` but stores `_splitToken` (line 108) without checking:

```
$._splitToken = _splitToken;
```

Additionally, entries in `FeeFlow`'s `_claimableTokens` array (line 134-136) are not validated before being added to the whitelist.

Neither `_bidToken` nor `_splitToken` has a setter — they are write-once at initialization. If either is set to an invalid address, the contract is permanently bricked: all `claim()` or `split()` calls would revert on the first `safeTransferFrom` or `balanceOf` call. Fixing this would require a contract upgrade.

Recommendation

Consider adding validation for all token parameters in both initializers:

```
  // FeeFlow.initialize()
  if (_destination == address(0)) revert FeeFlow_InvalidAddress();
  if (_bidThreshold < _minBidThreshold) revert FeeFlow_ThresholdBelowMin();
+ if (address(_bidToken) == address(0)) revert FeeFlow_InvalidAddress();
```

```
  // Splitter.initialize()
  if (_emergencyAdmin == address(0)) revert Splitter_InvalidAddress();
+ if (address(_splitToken) == address(0)) revert Splitter_InvalidAddress();
```

Informational Findings

[I-01] Redundant balance check in claim loop

Description

In `claim()`, the balance check inside the claim loop contains a redundant first clause:

```
uint256 _balance = _token.balanceOf(address(this));
if (_balance == 0 || _balance < _claimRequests[_i].minAmountRequested) {
    revert FeeFlow_InsufficientBalance();
}
```

The `_balance == 0` condition is subsumed by `_balance < _claimRequests[_i].minAmountRequested` for any `minAmountRequested >= 1`. The only case where it is load-bearing is when `minAmountRequested == 0`, where it prevents a zero-

amount `safeTransfer()`. That transfer is harmless (most ERC-20 implementations allow it), so the zero check serves no protective purpose and only adds gas cost and cognitive overhead.

Recommendation

Remove the redundant clause:

```
- if (_balance == 0 || _balance < _claimRequests[_i].minAmountRequested) {
+ if (_balance < _claimRequests[_i].minAmountRequested) {
      revert FeeFlow_InsufficientBalance();
  }
```

[I-02] Unsafe downcast of distributor weights in deploy script

Description

In `_distributorsFromEnv()`, the `DISTRIBUTOR_WEIGHTS` environment variable is parsed as `uint256[]`, and each element is explicitly cast to `uint96` when constructing `DistributorConfig` structs:

```
_distributors[_i] =
    Splitter.DistributorConfig({recipient: _recipients[_i], weight: uint96(_weights[
```

In Solidity 0.8, an explicit downcast from `uint256` to `uint96` silently truncates values that exceed `type(uint96).max` (~7.9e28). There is no overflow revert. If an operator enters a weight that overflows `uint96` – for example, `2**96 + 5` – the cast silently produces `5`, deploying with an unintended distribution ratio.

The existing `_weights[_i] == 0` check on line 163 catches the special case where the truncated value is exactly zero (e.g., `2**96` truncates to `0`), but any `2**96 + N` where `N > 0` passes all validation and deploys with weight `N`.

This is not exploitable by an external attacker. The in-scope `Splitter` contract receives weights already typed as `uint96` via ABI encoding, so the ABI decoder enforces the bound. The issue is limited to the operator-facing deploy script, which is technically out of audit scope but worth flagging.

Recommendation

Use OpenZeppelin's `SafeCast.toUint96()`, which reverts on overflow:

```
import {SafeCast} from "openzeppelin-contracts/contracts/utils/math/SafeCast.sol";
// ...
weight: SafeCast.toUint96(_weights[_i])
```

[I-03] Splitter lacks a recover function for stray tokens

Description

`FeeFlow` provides a `recover()` function (L237-241) gated on `DEFAULT_ADMIN_ROLE` that allows the admin to sweep any token to an arbitrary address:

```
function recover(IERC20 _token, address _to, uint256 _amount) external {
    _revertIfNotDefaultAdmin();
    _token.safeTransfer(_to, _amount);
    emit Recovered(_token, _to, _amount);
}
```

`Splitter` has no equivalent mechanism. Its `split()` function operates exclusively on `splitToken` via `balanceOf(address(this))` and never touches any other token. Any non-`splitToken` ERC-20 sent to the Splitter address — whether by user error, a misconfigured upstream system, or an incorrect integration — is permanently stuck with no recovery path.

`Splitter` already has the required access control infrastructure (`DEFAULT_ADMIN_ROLE`, `_revertIfNotDefaultAdmin()`) wired up via `AccessControlUpgradeable`, so adding a `recover()` function requires no new role plumbing. The project spec's Q&A section also explicitly added a "recover method" decision in response to the question about tokens falling into limbo during failed distributions.

Recommendation

Consider adding a `recover()` function to `Splitter` mirroring `FeeFlow`'s implementation:

```
event Recovered(IERC20 indexed token, address indexed to, uint256 amount);

function recover(IERC20 _token, address _to, uint256 _amount) external {
    _revertIfNotDefaultAdmin();
    SafeERC20.safeTransfer(_token, _to, _amount);
    emit Recovered(_token, _to, _amount);
}
```

[I-04] Distributor weights are unconstrained ratios instead of enforced percentages

Description

The `_setDistributors()` function accepts arbitrary positive integers as distributor weights and computes each recipient's share as a ratio of the individual weight to the sum of all weights:

```
uint256 _share = (_distributeAmount * $._distributors[_i].weight) / _totalWeight;
```

The spec states: "Must have a method for governance to set the distribution percentage (weight) for each distributor." The kickoff transcript consistently uses percentage language (e.g., "30% burn, 70% distribution... two distributors with 50% distributing each"), and the integration tests use percentage-shaped weights (`{weight: 50}` for two distributors summing to 100).

However, the code imposes no constraint on the sum of weights or the range of individual weights – only that each weight is non-zero. This means weights function as relative ratios rather than literal percentages:

- `[50, 50]` (sum 100) gives 50/50.
- `[1, 1]` (sum 2) also gives 50/50.
- `[50, 50, 50]` (sum 150) gives 33.3% each – not 50% each.

This creates a governance foot-gun. Consider a scenario where governance configures `[{A, 50}, {B, 50}]` intending "A gets 50%, B gets 50%." This works because the sum is 100. Later, governance adds `{C, 50}` intending "C gets 50%." The actual result is A/B/C each receiving 33.3% because the sum is now 150. Conversely, setting `{A, 1}` with the intent of "1%" produces a share that depends entirely on the other weights, not a fixed 1%.

Recommendation

If percentages are desired, enforce that weights are literal percentages by constraining each weight to `[1, 100]` and requiring their sum equals exactly 100.

Alternatively, if relational weights are desired, update user documentation, code comments, and spec to clearly highlight this behavior.

Gas Findings

[G-01] Cache storage reads in `claim` to avoid redundant SLOADs

Description

In `claim()`, the storage variables `$._destination` and `$._bidToken` are each read from ERC-7201 namespaced storage multiple times, with external calls intervening between reads. Because the Solidity compiler cannot optimize across external call boundaries, each access compiles to a separate SLOAD instruction.

```
$._bidToken.safeTransferFrom(msg.sender, $._destination, _bidAmount);
ISplitter($._destination).split();

for (uint256 _i = 0; _i < _claimRequests.length; _i++) {
    IERC20 _token = _claimRequests[_i].token;
    if (_token == $._bidToken) revert FeeFlow_InvalidFeeToken();
```

Specifically:

- `$._destination` is read twice: once on line 215 inside `safeTransferFrom()` and once on line 216 for the `ISplitter().split()` call. The external `safeTransferFrom()` call between the two reads prevents the compiler from caching the value.
- `$._bidToken` is read twice: once on line 215 for `safeTransferFrom()` and once on line 220 inside the loop for the `== $._bidToken` comparison. The intervening external calls again prevent compiler optimization. For multi-token claims, the loop re-read is a warm SLOAD on every iteration.

The estimated savings are approximately 4,200 gas from avoiding two cold SLOADs, plus roughly 100 gas per loop iteration for the warm `$._bidToken` re-read. There is no functional change.

Recommendation

Consider caching both `$._bidToken` and `$._destination` in local memory variables at the top of the function, alongside the existing `_bidAmount` cache:

```
  function claim(ClaimRequest[] calldata _claimRequests) external {
    FeeFlowStorage storage $ = _getFeeFlowStorage();
    if ($._claimPaused) revert FeeFlow_ClaimPaused();
    uint256 _bidAmount = $._bidThreshold;
+   IERC20 _bidToken = $._bidToken;
+   address _destination = $._destination;

-   $._bidToken.safeTransferFrom(msg.sender, $._destination, _bidAmount);
-   ISplitter($._destination).split();
+   _bidToken.safeTransferFrom(msg.sender, _destination, _bidAmount);
+   ISplitter(_destination).split();

    for (uint256 _i = 0; _i < _claimRequests.length; _i++) {
      IERC20 _token = _claimRequests[_i].token;
-     if (_token == $._bidToken) revert FeeFlow_InvalidFeeToken();
+     if (_token == _bidToken) revert FeeFlow_InvalidFeeToken();
```